

Canvas -> Plan -> Deploy

Material de apoyo para presentacion

Este documento reúne tres versiones complementarias para explicar el roadmap del sistema:

1. `Roadmap explicado de forma natural`
2. `Roadmap explicado para documento`
3. `Guion oral para presentarlo en vivo`

La idea es que sirva tanto para estudiar el flujo como para explicarlo en una demo, una demo o una presentación ante profesores.

1. Roadmap explicado de forma natural

La idea general del sistema es esta:

un estudiante o usuario diseña una red en un canvas visual, el backend toma ese diseño, lo convierte en algo técnicamente entendible, lo valida, lo guarda como un plan y, si todo está bien, puede lanzarlo para desplegar infraestructura real en la nube.

Dicho más simple:

`dibujo en el canvas -> plan técnico -> validación -> ejecución -> resultado`

1.1 Todo empieza en el canvas

El usuario no trabaja escribiendo Terraform ni configuraciones complejas desde el inicio. Empieza con algo visual: dibuja su topología, define segmentos, subredes, conexiones y recursos.

Eso hace que la experiencia sea más intuitiva, especialmente en un contexto académico, porque el punto de entrada es más cercano al aprendizaje visual que a la automatización pura.

Pero aquí está la parte importante: el sistema no trata ese canvas como una imagen. Lo trata como una intención técnica.

Es decir, lo que el usuario dibuja no se guarda solo para verlo bonito, sino para convertirlo en una definición de red que el backend pueda procesar.

1.2 El backend recibe ese diseño y lo limpia

Cuando el canvas se sincroniza con el backend, entra un payload con la información del laboratorio.

Lo primero que hace el sistema es limpiarlo y normalizarlo. Eso significa:

- revisar que venga en un formato válido
- corregir nombres o alias viejos
- asegurarse de que exista un identificador único del canvas
- dejar todo en una estructura consistente

La idea aquí es evitar que el backend dependa de cómo exactamente lo manda el frontend. Aunque la entrada tenga variantes, internamente el sistema intenta dejar un formato más estable y predecible.

1.3 El sistema convierte el diseño en una intención técnica común

Despues de limpiar la entrada, el backend traduce ese payload a un lenguaje intermedio, una especie de modelo neutral de red.

Esto es importante porque separa dos cosas:

- lo que el usuario quiso disenar
- la forma especifica en que eso se ejecuta en una nube concreta

En otras palabras, primero se entiende la intencion:

"quiero esta red, con estos segmentos, estas conexiones y estos recursos"

y luego se piensa:

"como represento eso para AWS?"

Ese paso ayuda mucho a que la arquitectura tenga orden y pueda crecer despues.

1.4 El adapter del provider traduce eso al formato real de ejecucion

Una vez que el sistema ya entendio la intencion, entra el adapter del provider.

Aqui es donde el backend dice:

"bien, ya entendi la red que el usuario quiere; ahora la voy a convertir al formato exacto que AWS necesita".

Esto es lo que llamamos compilar el payload.

No significa desplegar todavia.

Significa traducir desde un lenguaje mas flexible o mas visual a un lenguaje mas estricto y tecnico.

Ese resultado final ya tiene la forma que espera el runtime de Terraform.

1.5 El plan se guarda como una unidad tecnica

Despues de validar y compilar, el backend crea o actualiza un `Plan`.

Ese plan es importante porque ya no estamos hablando solo del canvas original, sino de una entidad tecnica persistida.

El canvas es el punto de partida visual.

El plan es la version operativa y desplegable de ese canvas.

Ahi queda guardado:

- el payload ya procesado
- el estado actual
- la relacion con el laboratorio
- el identificador del canvas
- y luego, cuando se ejecute, tambien logs, outputs y contexto de ejecucion

1.6 Cuando el usuario pide deploy, no se ejecuta directo

Cuando alguien presiona desplegar, el backend no corre Terraform inmediatamente dentro del request web.

Primero hace varios checks:

- verifica que el plan exista y que el usuario lo pueda ver
- revisa que no haya otra ejecucion corriendo
- comprueba permisos
- revisa con que cuenta cloud se ejecutaria

- valida que no haya inconsistencias con ejecuciones anteriores
- revalida el payload antes de lanzarlo

Esto es una parte importante para explicar el valor del sistema: no es un boton que lanza infraestructura de forma ciega. Hay control, reglas y seguridad antes de ejecutar.

1.7 Celery toma el trabajo pesado

Si todo esta correcto, la vista web no ejecuta Terraform directamente.

Lo que hace es encolar una tarea en Celery.

Esto significa que el request responde rapido y la parte pesada se va a un worker en segundo plano.

La logica es:

- la web prepara el trabajo
- Celery lo toma
- el worker ejecuta la tarea real

Eso permite que la aplicacion no se bloquee mientras corre el despliegue.

1.8 La tarea prepara el entorno real de ejecucion

Cuando Celery toma la tarea, empieza la parte mas tecnica.

La tarea:

- carga el plan
- detecta el provider
- resuelve la cuenta cloud
- prepara credenciales
- arma un bundle de ejecucion
- crea un workspace temporal
- renderiza el main.tf
- prepara el state local
- y luego ejecuta Terraform

Ese bundle es como el paquete de trabajo de la ejecucion: contiene todo lo necesario para correr el plan correctamente.

1.9 Terraform ejecuta el flujo real

Ya en esta fase, el sistema corre los comandos reales:

- `terraform init`
- `terraform plan`
- `terraform apply`

Aqui si estamos en la parte donde la intencion del usuario se convierte en infraestructura real.

Eso ya no es simulacion conceptual. Es ejecucion concreta.

Por eso todo lo anterior importa tanto: porque llegar a este punto sin validacion o sin control seria riesgoso.

1.10 El sistema guarda el resultado y deja trazabilidad

Cuando la ejecucion termina, el sistema actualiza el plan.

Si salio bien:

- marca `SUCCESS`
- guarda outputs
- registra contexto de ejecucion
- conserva logs

Si salio mal:

- marca `FAILURE`
- guarda el error
- conserva logs para depuracion

Esto es muy valioso para explicar el proyecto porque muestra que no es solo automatizacion, sino tambien trazabilidad.

Se puede saber:

- que se intento hacer
- quien lo lanzo
- con que cuenta cloud
- que devolvio Terraform
- y en que estado quedo

1.11 Si algo queda pegado, el sistema puede reconciliarlo

Hay una funcion de reconciliacion para los planes que quedaron en `RUNNING` pero cuya tarea ya termino o se quedo stale.

Eso ayuda a mantener consistencia entre:

- lo que Celery sabe
- y lo que la base de datos cree que esta pasando

Es decir, el sistema tambien se preocupa por corregir estados incoherentes.

1.12 Version resumida para explicarlo en voz alta

Puedes decirlo asi:

"Primero el usuario disena una red en el canvas. Ese diseno no se guarda solo como una imagen, sino como una intencion tecnica. El backend toma esa intencion, la limpia, la valida y la traduce al formato exacto que AWS necesita. Luego la guarda como un plan. Cuando el usuario pide desplegar, el sistema no ejecuta de inmediato, sino que primero revisa permisos, cuenta cloud y consistencia del estado. Si todo esta bien, encola una tarea en Celery. Esa tarea prepara el entorno, genera el Terraform y ejecuta el plan o el apply. Finalmente, el sistema guarda logs, outputs y estado final para que todo quede trazable y auditable."

1.13 Version aun mas simple

"Lo que el estudiante dibuja en el canvas se convierte en un plan tecnico, ese plan se valida y, si todo esta correcto, el sistema lo ejecuta automaticamente en la nube y guarda todo el resultado."

1.14 Idea clave para venderlo

"El valor del proyecto no es solo dibujar redes, sino convertir ese diseno en una experiencia real de validacion, automatizacion, despliegue y aprendizaje tecnologico."

2. Roadmap explicado para documento

2.1 Roadmap del flujo Canvas -> Plan -> Deploy

Este proyecto esta pensado para que un estudiante no se quede solo en dibujar un a red, sino que pueda llevar ese diseno a un proceso tecnico real. La plataforma toma lo que el usuario modela en un canvas visual, lo convierte en una definicion entendible por el sistema, lo valida, lo guarda como un plan y, si todo esta correcto, lo ejecuta en la nube.

La logica completa puede entenderse como este recorrido:

`canvas visual -> intencion tecnica -> plan validado -> ejecucion automatica -> resultado auditable`

2.2 El usuario comienza en el canvas

Todo empieza en la interfaz visual. El estudiante arma una topologia de red desde un canvas, conectando componentes y definiendo la estructura que quiere representar.

Lo importante aqui es que el canvas no se trata como una imagen o un simple dibujo. El sistema interpreta ese diseno como una propuesta tecnica. Es decir, el usuario esta modelando una red que luego podra ser procesada por el backend.

2.3 El backend recibe y organiza la informacion

Cuando el usuario guarda o sincroniza el canvas, el backend recibe un payload con toda la informacion de esa topologia.

Lo primero que hace el sistema es ordenar esa entrada:

- valida que tenga una estructura aceptable
- corrige nombres heredados o alias antiguos
- identifica el `canvas\_id`
- deja el contenido en un formato interno mas limpio

Este paso es importante porque permite que distintas versiones o formas de entrada terminen siendo tratadas de manera consistente.

2.4 El diseno se convierte en una intencion tecnica

Despues de limpiar la entrada, el sistema transforma el contenido en una especie de modelo neutral de red.

Este modelo representa lo que el usuario quiso construir, pero todavia sin depender completamente de una nube especifica. Es una forma de separar la intencion del usuario de la implementacion concreta.

Asi, el sistema primero entiende:

- que red quiere el usuario
- que segmentos tiene
- como se conectan
- que recursos forman parte del diseno

2.5 El adapter traduce esa intencion al lenguaje de AWS

Una vez que la intencion esta clara, entra el adapter del provider.

El adapter toma esa definicion mas abstracta y la convierte al formato exacto que AWS y Terraform necesitan para ejecutarla. A esto se le llama compilar el payload.

Aqui no se despliega todavia nada. Lo que ocurre es una traduccion tecnica:

- de un lenguaje mas flexible y cercano al canvas
- a un formato mas estricto y listo para ejecucion

Este paso es clave porque conecta el mundo visual con el mundo operativo.

2.6 El sistema guarda un plan

Despues de validar y compilar, el backend crea o actualiza un `Plan`.

Ese plan es la unidad tecnica desplegable del sistema. Ya no estamos hablando solo del canvas original, sino de una representacion persistida y operativa.

El plan guarda:

- el contenido procesado
- su estado
- el `canvas\_id`
- la relacion con el laboratorio
- y luego tambien logs, outputs e historial de ejecucion

2.7 Cuando se solicita deploy, el sistema primero revisa reglas

Cuando el usuario pide desplegar, el backend no ejecuta infraestructura inmediatamente.

Antes de lanzar nada, revisa:

- que el plan exista
- que no haya otra ejecucion corriendo
- que el usuario tenga permisos
- que la cuenta cloud sea la correcta
- que no haya inconsistencias con un deploy previo
- que el payload siga siendo valido

Esto es importante porque demuestra que la plataforma no es solo un boton de ejecucion, sino un flujo controlado y seguro.

2.8 Celery toma la ejecucion en segundo plano

Si todas las validaciones salen bien, la aplicacion web encola una tarea en Celery.

Eso significa que:

- la peticion web no se queda esperando
- el trabajo pesado se mueve a un worker asincrono
- la experiencia del usuario sigue siendo fluida

Celery actua como el encargado de ejecutar el proceso de fondo.

2.9 La tarea prepara el entorno real

Cuando el worker toma la tarea, empieza la ejecucion tecnica real.

La tarea:

- carga el plan
- detecta el provider
- resuelve credenciales y cuenta cloud
- construye un contexto de ejecucion
- prepara un workspace temporal
- renderiza el archivo Terraform
- y deja listo el entorno para correr comandos reales

2.10 Terraform ejecuta la infraestructura

En esta etapa el sistema llama los comandos reales:

- `terraform init`
- `terraform plan`
- `terraform apply`

Aqui es donde la intencion del usuario se convierte en infraestructura real.

El valor del proyecto esta justamente en este punto: el estudiante no solo disen a una topologia, sino que participa en un flujo que puede materializarla tecnica mente.

2.11 El resultado queda guardado y trazable

Al terminar la ejecucion, el sistema actualiza el plan.

Si todo sale bien:

- marca `SUCCESS`
- guarda outputs
- conserva logs
- registra el contexto de la ejecucion

Si algo falla:

- marca `FAILURE`
- guarda el error
- conserva logs para analisis

Esto permite trazabilidad completa y facilita tanto la evaluacion academica como la depuracion tecnica.

2.12 El sistema tambien corrige estados inconsistentes

Si una tarea quedo trabada o un plan quedo marcado como `RUNNING` cuando en realidad ya termino, el sistema puede reconciliar ese estado.

Eso ayuda a mantener consistencia entre:

- lo que ocurrio realmente en Celery
- y lo que quedo guardado en base de datos

2.13 Conclusiones

En resumen, el proyecto convierte un diseno visual en una experiencia tecnica completa.

El estudiante:

- disena
- valida
- despliega

- revisa resultados

Y el profesor puede observar no solo el resultado final, sino todo el proceso que llevo a el.

3. Guion oral para presentarlo en vivo

"Lo que hace esta plataforma es tomar algo que normalmente se queda en un dibujo, como una topologia de red en un canvas, y convertirlo en un proceso tecnico real.

El estudiante empieza disenando visualmente su laboratorio. Pero ese diseno no se guarda solo como una imagen, sino como una intencion tecnica. Es decir, el sistema intenta entender que red quiso construir, que componentes tiene y como deberian relacionarse.

Despues, el backend toma esa informacion, la limpia y la organiza. Luego la traduce a un formato mas estricto que ya puede ser entendido por AWS y por Terraform. Ese paso es importante porque conecta lo visual con lo ejecutable.

Una vez hecho eso, el sistema guarda un plan. Ese plan ya es una unidad tecnica sobre la que se puede trabajar. No es solo el canvas original, sino una version validada y preparada para despliegue.

Cuando el usuario pide ejecutar, el sistema no lo hace de manera ciega. Primero revisa permisos, estado del plan, cuenta cloud y consistencia de la ejecucion. Si todo esta correcto, encola una tarea en Celery.

Celery toma el trabajo pesado en segundo plano, prepara el entorno, genera el Terraform y ejecuta los comandos necesarios, como `terraform init`, `terraform plan` y `terraform apply`.

Al final, la plataforma guarda el resultado completo: estado final, logs, outputs e historial. Eso permite que el estudiante vea que ocurrio y que el profesor pueda evaluar no solo el resultado, sino todo el proceso.

En pocas palabras, el valor del proyecto esta en que el estudiante no solo dibuja una arquitectura, sino que aprende como esa arquitectura se valida, se automatiza y se lleva a un entorno real."

4. Idea de cierre para presentacion

Puedes cerrar con una frase como esta:

"El valor de esta plataforma no esta solo en representar redes, sino en convertir ese diseno en una experiencia completa de validacion, automatizacion, despliegue y aprendizaje tecnico."